40. Given four lists A, B, C, D of integer values,Write a program to compute how many tuples n(i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is zero. (i) Input: A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2] Output: 2 (ii) Input: A = [0], B = [0], C = [0], D = [0] Output: 1

```
from collections import Counter

A = [1,2]
B = [-2,-1]
C = [-1,2]
D = [0,2]

ab = Counter(a+b for a in A for b in B)

count = 0
for c in C:
    for d in D:
        count += ab[-(c+d)]

print(count)
```

39. Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0). (i) Input : points = [[1,3],[-2,2],[5,8],[0,1]],k=2 Output:[[-2, 2], [0, 1]] (ii) Input: points = [[1, 3], [-2, 2]], k = 1 Output: [[-2, 2]] (iii) Input: points = [[3, 3], [5, -1], [-2, 4]], k = 2 Output: [[3, 3], [-2, 4]]

```
points = [[1,3],[-2,2],[5,8],[0,1]]
k = 2

points.sort(key=lambda p: p[0]**2 + p[1]**2)
print(points[:k])
```

```
[[0, 1], [-2, 2]]
```

38. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm? Input : N= 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31 Output : 6 Test cases Input : N= 7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7 Test cases Input : N= 6, a[] = {20,40,60,80,100,120}, search key = 60 Output : 3

```
a = [3,9,14,19,25,31,42,47,53]
key = 31

l, r = 0, len(a)-1

while l <= r:
    mid = (l+r)//2
    print("Mid =", mid+1, "Value =", a[mid])

    if a[mid] == key:
        print("Position =", mid+1)
        break
    elif a[mid] < key:
        l = mid+1
    else:
        r = mid-1
```

```
Mid = 5 Value = 25
Mid = 7 Value = 42
Mid = 6 Value = 31
Position = 6
```

37. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result. Input : N= 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20 Output : 4 Test cases Input : N= 6, a[] = {10,20,30,40,50,60}, search key = 50 Output : 5 Input : N= 7, a[] = {21,32,40,54,65,76,87}, search key = 32 Output : 2

```
a = [5,10,15,20,25,30,35,40,45]
key = 20

l, r = 0, len(a)-1
count = 0

while l <= r:
    count += 1
    mid = (l+r)//2

    if a[mid] == key:
        print("Position =", mid+1)
        print("Comparisons =", count)
        break
    elif a[mid] < key:
        l = mid+1
    else:
        r = mid-1
```

```
Position = 4
Comparisons = 4
```

36. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array. Input : N= 8, a[] = {19,72,35,46,58,91,22,31} Output : 19,22,31,35,46,58,72,91 Test Cases : Input : N= 8, a[] = {31,23,35,27,11,21,15,28} Output : 11,15,21,23,27,28,31,35 Test Cases : Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17} Output : 13,17,22,25,34,36,43,52,65,67

```
def quick_sort(a):
    if len(a) <= 1:
        return a

    pivot = a[len(a)//2]
    left = [x for x in a if x < pivot]
    mid = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]

    return quick_sort(left) + mid + quick_sort(right)

a = [19,72,35,46,58,91,22,31]
print(*quick_sort(a))
```

```
19 22 31 35 46 58 72 91
```

35. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array

after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Input : N= 9, a[]= {10,16,8,12,15,6,3,9,5} Output : 3,5,6,8,9,10,12,15,16 Test Cases : Input : N= 8, a[] = {12,4,78,23,45,67,89,1} Output : 1,4,12,23,45,67,78,89 Test Cases : Input : N= 7, a[] = {38,27,43,3,9,82,10} Output : 3,9,10,27,38,43,82

```
def quick_sort(a):
    if len(a) <= 1:
        return a

    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

a = [10,16,8,12,15,6,3,9,5]
print(*quick_sort(a))
```

```
3 5 6 8 9 10 12 15 16
```

34. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array. Test Cases : Input : N= 8, a[] = {12,4,78,23,45,67,89,1} Output : 1,4,12,23,45,67,78,89 Test Cases : Input : N= 7, a[] = {38,27,43,3,9,82,10} Output : 3,9,10,27,38,43,82

```
count = 0

def merge_sort(a):
    global count
    if len(a) > 1:
        m = len(a)//2
        L, R = a[:m], a[m:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0
        while i < len(L) and j < len(R):
            count += 1
            if L[i] < R[j]:
                a[k] = L[i]
                i += 1
            else:
                a[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            a[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            a[k] = R[j]
            j += 1
            k += 1

a = [12,4,78,23,45,67,89,1]
merge_sort(a)
print("Sorted:", *a)
print("Comparisons:", count)
```

```
Sorted: 1 4 12 23 45 67 78 89
Comparisons: 16
```

33. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice. Test Cases : Input : N= 8, a[] = {31,23,35,27,11,21,15,28} Output : 11,15,21,23,27,28,31,35 Test Cases : Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17} Output : 13,17,22,25,34,36,43,52,65,67

```
def merge_sort(a):
    if len(a) > 1:
        m = len(a)//2
        L, R = a[:m], a[m:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                a[k] = L[i]
                i += 1
            else:
                a[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            a[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            a[k] = R[j]
            j += 1
            k += 1

a = [31,23,35,27,11,21,15,28]
merge_sort(a)
print(*a)
```

```
11 15 21 23 27 28 31 35
```

32. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found. Input : N=8, 2,4,6,8,10,12,14,18. Output : Min = 2, Max =18 Test Cases : Input : N= 9, a[] = {11,13,15,17,19,21,23,35,37} Output : Min = 11, Max = 37 Test Cases : Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17} Output : Min 12, Max 67

```
a = [2,4,6,8,10,12,14,18]
print("Min =", a[0])
print("Max =", a[-1])
```

```
Min = 2
Max = 18
```

31. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found. Input : N= 8, a[] = {5,7,3,4,9,12,6,2} Output : Min = 2, Max = 12 Test Cases : Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17} Output : Min = 1, Max = 17 Test Cases : Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17} Output : Min 12, Max 67

```
a = [5,7,3,4,9,12,6,2]
print("Min =", min(a))
print("Max =", max(a))
```

```
Min = 2
Max = 12
```

30. Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product Z=X x Y Test Case 1: Input: x=1234,y=5678 Expected Output: z=1234×5678=7016652

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    high1, low1 = divmod(x, 10**m)
    high2, low2 = divmod(y, 10**m)

    z0 = karatsuba(low1, low2)
    z1 = karatsuba((low1+high1), (low2+high2))
    z2 = karatsuba(high1, high2)

    return (z2 * 10**(2*m)) + ((z1-z2-z0) * 10**m) + z0

x = 1234
y = 5678

print(karatsuba(x,y))
```

```
7006652
```

29. Given two 2×2 Matrices A and B A=(1 7 B=(1 3 3 5) 7 5) Use Strassen's matrix multiplication algorithm to compute the product matrix C such that C=A×B. Test Cases: Consider the following matrices for testing your implementation: Test Case 1: A=(1 7 B=(6 8 3 5), 4 2) Expected Output: C=(18 14 62 66)

```python
def strassen_2x2(A, B):
    a,b = A[0]
    c,d = A[1]

    e,f = B[0]
    g,h = B[1]

    M1 = (a+d)*(e+h)
    M2 = (c+d)*e
    M3 = a*(f-h)
    M4 = d*(g-e)
    M5 = (a+b)*h
    M6 = (c-a)*(e+f)
    M7 = (b-d)*(g+h)

    C11 = M1+M4-M5+M7
    C12 = M3+M5
    C21 = M2+M4
    C22 = M1-M2+M3+M6

    return [[C11,C12],[C21,C22]]

A = [[1,7],[3,5]]
B = [[6,8],[4,2]]

print(strassen_2x2(A,B))
```

```
[[34, 22], [38, 34]]
```

28. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false. a) E = {1, 3, 9, 2, 7, 12} exact Sum = 15 b) E = {3, 34, 4, 12, 5, 2} exact Sum = 15

```
from itertools import combinations
def subset_sums(arr):
    sums = []
    for r in range(len(arr)+1):
        for comb in combinations(arr, r):
            sums.append(sum(comb))
    return sums
def exact_subset_sum(arr, target):
    n = len(arr)
    left = arr[:n//2]
    right = arr[n//2:]
    left_sums = subset_sums(left)
    right_sums = set(subset_sums(right))
    for s in left_sums:
        if target - s in right_sums:
            return True
    return False
print(exact_subset_sum([1,3,9,2,7,12],15))
print(exact_subset_sum([3,34,4,12,5,2],15))
```

```
True
True
```

27. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset. a) Set[] = {45, 34, 4, 12, 5, 2} Target Sum : 42 b) Set[]= {1, 3, 2, 7, 4, 6} Target sum = 10:

```
from itertools import combinations
import bisect
def subset_sums(arr):
    sums = []
    for r in range(len(arr)+1):
        for comb in combinations(arr, r):
            sums.append(sum(comb))
    return sums
def closest_subset_sum(arr, target):
    n = len(arr)
    left = arr[:n//2]
    right = arr[n//2:]
    left_sums = subset_sums(left)
    right_sums = sorted(subset_sums(right))
    best = float('inf')
    for s in left_sums:
        idx = bisect.bisect_left(right_sums, target-s)
        if idx < len(right_sums):
            best = min(best, abs(target-(s+right_sums[idx])))

        if idx > 0:
            best = min(best, abs(target-(s+right_sums[idx-1])))
    return target - best
print(closest_subset_sum([45,34,4,12,5,2],42))
print(closest_subset_sum([1,3,2,7,4,6],10))
```

```
41
10
```

26. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array. arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6 arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] k = 5 Output: An integer representing the k-th smallest element in the array.

```
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]
    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = median_of_medians(medians, len(medians)//2 + 1)
    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    equal = [x for x in arr if x == pivot]
    if k <= len(low):
        return median_of_medians(low, k)
    elif k <= len(low) + len(equal):
        return pivot
    else:
        return median_of_medians(high, k - len(low) - len(equal))
print(median_of_medians([1,2,3,4,5,6,7,8,9,10],6))
print(median_of_medians([23,17,31,44,55,21,20,18,19,27],5))
```

```
6
21
```

CHAPTER - 3

25. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array. arr = [12, 3, 5, 7, 19] k = 2 Expected Output:5 arr = [12, 3, 5, 7, 4, 19, 26] k = 3 Expected Output:5 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6 Expected Output:6

```python
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]
    groups = [arr[i:i + 5] for i in range(0, len(arr), 5)]
    medians = [sorted(group)[len(group)//2] for group in groups]
    pivot = median_of_medians(medians, len(medians)//2 + 1)
    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    equal = [x for x in arr if x == pivot]
    if k <= len(low):
        return median_of_medians(low, k)
    elif k <= len(low) + len(equal):
        return pivot
    else:
        return median_of_medians(high, k - len(low) - len(equal))
print(median_of_medians([12,3,5,7,19], 2))
print(median_of_medians([12,3,5,7,4,19,26], 3))
print(median_of_medians([1,2,3,4,5,6,7,8,9,10], 6))
```

```
5
5
6
```